

Prática #05 - 2025.1

Objetivos

Refazer o teste do multiplicador de 4 bits da primeira prática e integrar a comunicação PC $\leftrightarrow$ HPS $\leftrightarrow$ FPGA, agora utilizando uma Dual-Port-RAM feita por nós e integrada com o controle do multiplicador.

Instruções Gerais

- Como entrega para esta atividade:
 - mostrar o sistema funcionando;
 - colocar no classrrom como resposta o link para um repositório no seu github com o projeto feito (contendo o sistema separado por pastas - PC, HPS e FPGA).

1 O Sistema

Nosso sistema de testes deve ser configurado como mostrado no diagrama de blocos da Figura 1:

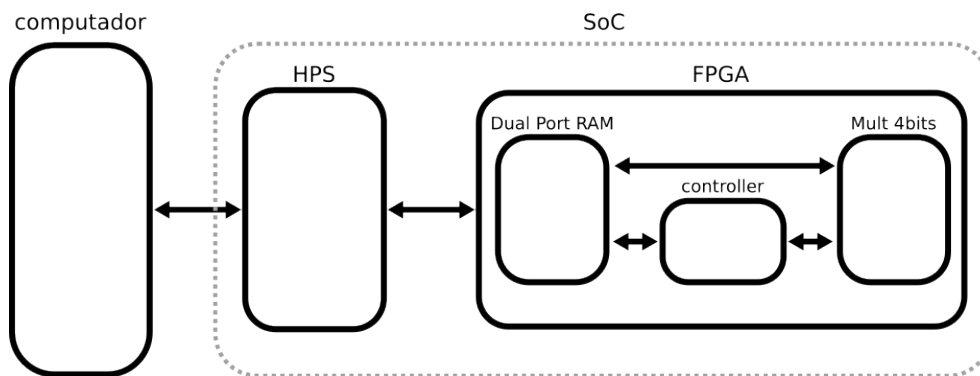


Figura 1: Diagrama de blocos do sistema de testes.

- **Computador:** *script* Python responsável por gerar as entradas do sistema, enviá-las para o **SoC**, receber as respostas enviadas pelo **SoC** e verificá-las;
- **SoC:**
 - **HPS:** programa em C responsável por receber as entradas vindo do **Computador**, tratá-las e alimentar a **Dual Port RAM**. Controlar o início e verificar o fim do cálculo feito pelo **Mult 4bits**. Após o término do cálculo, enviar a resposta para o **Computador**;

– **FPGA:**

* **Dual-Port RAM:** um módulo feito por nós possuindo duas interfaces:.

1. **wishbone SLAVE:** interface a ser ligada ao HPS através do *IP core* Av-MM to Wishbone Bridge, criado na última aula;
2. **custom:** entradas e saídas dos sinais A, B, Y, **enable** e **fim**;

Uma vez que estamos construindo nossa memória, podemos criá-la com apenas uma palavra de 32 bits. Uma sugestão de função relacionada a cada byte da memória é dada na Tabela 1.

* **Mult 4bits:** módulo criado na Prática 1;

* **controller:** módulo a ser criado responsável por lidar com os sinais A, B, Y, **enable** e **fim**.

Tabela 1: Sugestão de função dos bytes guardados na **Dual Port RAM**.

sel_i	Nome	Funções
0001	<i>control</i>	bit[0] = enable
0010	<i>data in</i>	bit[3:0] = A; bit[7:4] = B
0100	<i>data out</i>	bit[7:0] = Y
1000	<i>status</i>	bit[0] = fim